

1. Daily checks

Start here: Is the system healthy? What is this pod doing? When is its next task?

Run from the repository root in PowerShell. pod_dv2_01 is an example: replace it with your enabled pod. Examples use paper mode; live uses real money. Keep the same release root and existing DB as the running service. Append --releases-root and --db-path when your service uses them. Never start an existing trading pod with a new empty DB.

Terms: pod = one strategy and its account state; ACK = broker acknowledgment; fill = executed trade; NetLiq = total account value. A backtick at the end of a command continues it on the next line.

READ = read-only report. INSPECT = no order submission, but may write diagnostic records. ACTIVE = changes state, data or services. All values below are illustrative; output is shortened.

Check overall health

READ

Shows all enabled pods in the selected mode.

```
uv run python -m alpha.live.runner ops_report --mode paper
```

Example output - selected lines

```
Mode: paper
Generated: 2026-09-17T12:00:00+00:00
Source as-of: 2026-09-17T11:59:30+00:00
Overall: GREEN
```

GREEN: fresh evidence passed. YELLOW: expected waiting/review. RED: investigate. GRAY: evidence is missing. Check timestamps before trusting the color.

Check one pod

INSPECT

Shows current state, next action, broker ACK gaps and execution exceptions.

```
uv run python -m alpha.live.runner status --mode paper --pod-id pod_dv2_01
```

Example output - selected lines

```
- Auto-submit: off
- State: order plan ready
- Next: review orders manually
- Reason: execution plan is ready to submit
```

This pod has orders ready for manual review. It has not proved that orders were filled. Check the account, timing and order plan before any submission.

Check the next scheduled task

INSPECT

Explains when the scheduler plans to wake or act.

```
uv run python -m alpha.live.scheduler_service next_due --mode paper --pod-id pod_dv2_01
```

Example output - selected lines

```
- Due now: false
- Phase: idle_probe
- Reason: no_due_work
- Next due UTC: 2026-09-17T12:01:00+00:00
```

Here, no trading task is due. The next timestamp is a check time, not necessarily an order time. Timestamps labeled UTC are not local Israel time.

2. Diagnose and review plans

DecisionPlan = what the strategy wants. VPlan = exact orders sized from current broker state.

Find why a pod is blocked

ACTIVE

doctor is for paper/live. It queries the broker and may sync data. Use an unused broker client ID.

```
uv run python -m alpha.live.runner doctor --mode paper --pod-id pod_dv2_01 --broker-client-id 33 --json
```

Example JSON excerpt - inspect the component list

```
{
  "overall_verdict_str": "BLOCK",
  "component_result_dict_list": [{
    "component_name_str": "broker",
    "status_str": "BLOCK",
    "reason_code_str": "account_not_visible",
    "detail_str": "Expected account 'EXAMPLE' is not visible."
  }]
}
```

33 is only an example; verify it is free. PASS = checks passed; WAIT = expected condition pending; BLOCK = fix the failed component. Here, check the configured account route and the account logged into TWS/Gateway. Do not run doctor in a loop.

Read the strategy decision

INSPECT

Check the signal time, execution policy, targets and planned execution time.

```
uv run python -m alpha.live.runner show_decision_plan --mode paper --pod-id pod_dv2_01
```

Example output - selected lines

```
DecisionPlan
Pod: pod_dv2_01
- DecisionPlan id: 41
- Status: planned
- Execution policy: next_open_moo
- Target: SPY | weight=0.100000
```

This example wants a 10% target weight. It is strategy intent, not a fill or a final share count. If no plan is found, check status and next_due before trying to build one.

Read the exact order plan

INSPECT

Review the plan ID, account in the decision, budget, prices, share changes and warnings.

```
uv run python -m alpha.live.runner show_vplan --mode paper --pod-id pod_dv2_01
```

Example output - selected lines

```
VPlan
Pod: pod_dv2_01
- VPlan id: 42
- DecisionPlan id: 41
- Status: ready
- NetLiq: 100000.0
- Pod budget: 100000.0
- Plan: SPY | current=0.0000 | target=20.0000 | delta=20.0000 | ...
```

ready means prepared, not submitted. This row adds 20 SPY shares. The full row also shows the reference price, its source and estimated value. After submission, check Submit ACK, missing responses and broker-vs-target residuals.

Manual review flow - only when auto-submit is off:

```
[tick] -> [review decision + VPlan] -> [submit] -> [fills] -> [reconcile]
```

3. tick, run_once and serve

All three can advance trading. In live mode, they may send real orders when auto-submit is enabled.

tick: run one trading pass

ACTIVE

Runs work that is due now. It may build plans, submit orders and reconcile.

```
uv run python -m alpha.live.runner tick --mode paper --pod-id pod_dv2_01
```

Example output - selected lines

```
Tick Result
- No sleeve needed action in this run.
- Built 0 decision plans.
- Built 0 order plans.
- Submitted 0 order plans.
- Completed 0 reconciliations.
- Blocked actions: 0
- Warnings: 0
```

Zero counts can be normal: nothing was due. If a plan was built, review it. If orders were submitted, check ACK and fills. A blocked action or warning needs investigation.

run_once: one scheduler pass

ACTIVE

Uses scheduler timing and task selection once, then exits.

```
uv run python -m alpha.live.scheduler_service run_once --mode paper --pod-id pod_dv2_01
```

Example output - selected lines

```
Scheduler
- Due now: false
- Phase: idle_probe
- Reason: no_due_work
- Tick invoked: false
```

Here the scheduler chose not to call tick because nothing was due. If Tick invoked is true, also read the Tick summary and post-tick pod state.

serve: keep the scheduler running

ACTIVE

Waits, runs due work, and waits again. Use the approved service command and existing DB.

```
uv run python -m alpha.live.scheduler_service serve --mode paper --pod-id pod_dv2_01
```

Typical event names - shortened, conditional sequence

```
service.start ... mode=paper pod=pod_dv2_01
cycle.wait ...
# Later, only when work is due:
cycle.start ...
build_vplan.ok ...
submit_vplan.ok ...
```

The process stays running. A start or wait message is not proof of a trade. Confirm ACK, fills and reconciliation separately. Do not start a second copy for a pod already served by a running service.

Quick choice: inspect timing with next_due; run one trading pass with tick; test one scheduler pass with run_once; keep the service alive with serve. Opening Dashboard V3 does not start serve.

4. Submit and confirm execution

A submitted plan is not a completed trade. Follow the evidence through ACK, fills and reconciliation.

Submit a reviewed VPlan

ACTIVE

Replace 42 with the current reviewed plan ID. Confirm the account and execution window.

```
uv run python -m alpha.live.runner submit_vplan --mode paper --pod-id pod_dv2_01 --vplan-id 42
```

Example output - selected lines

```
{
  "submitted_vplan_count_int": 1,
  "blocked_action_count_int": 0,
  "warning_count_map_dict": {},
  "reason_count_map_dict": {}
}
```

One plan was submitted in this example. That count alone does not prove all broker ACKs or fills. Check show_vplan and status before considering another submission.

Reconcile broker results

ACTIVE

Reads broker fills and positions, then updates the pod state.

```
uv run python -m alpha.live.runner post_execution_reconcile --mode paper --pod-id pod_dv2_01
```

Example output - selected lines

```
{"completed_vplan_count_int": 1}
```

One plan passed reconciliation and completed. A count of 0 can mean no eligible plan, still waiting, or unresolved differences. Read status and show_vplan; do not assume success.

Inspect fill evidence

INSPECT

Shows actual fill records and official open prices when available.

```
uv run python -m alpha.live.runner execution_report --mode paper --pod-id pod_dv2_01
```

Example output - selected lines

```
Execution Report
- VPlan id: 42
- Fill count: 2
- Fills with official open: 2
- Aggregate slippage notional: 3.5000
```

The full output lists each fill. If official open data is missing, auction slippage is not established. "No fills were found yet" may mean pending orders or missing evidence.

Record end-of-day account state

ACTIVE

After the session closes, saves broker cash, positions and NetLiq.

```
uv run python -m alpha.live.runner eod_snapshot --mode paper --pod-id pod_dv2_01
```

Example output - selected lines

```
EOD Snapshot
- Recorded snapshots: 1
- Skipped snapshots: 0
- Blocked actions: 0
```

Recorded = a snapshot was saved. Skipped can mean one already exists; read Reasons. This is account evidence, not a new trading decision.

5. Data, watchdog and support

Check the failed layer first. Data sync, monitoring and broker diagnostics are separate actions.

Read the saved monitoring report

READ

Reads the last report and the Windows task result; does not run the watchdog.

```
Get-Content alpha/live/logs/ops_report_latest.json
Get-ScheduledTaskInfo -TaskName AlphaLiveOpsWatchdog
```

Example excerpts from two separate outputs

```
"overall_severity_str": "green"
"source_stale_bool": false
LastTaskResult : 0
```

Also check report timestamps and LastRunTime. Watchdog codes: 0 = non-red result, 1 = red, 2 = fatal. Code 0 can include yellow or gray; it does not by itself prove health.

Run the watchdog once

ACTIVE

Writes the report and may send alerts and a heartbeat. It does not submit orders.

```
uv run python scripts/live_ops_watchdog.py --mode live --json
```

Example output - selected lines

```
"status_str": "ok"
"overall_severity_str": "green"
"notification_fired_count_int": 0
"heartbeat_status_str": "sent"
```

Check the heartbeat separately: failed or disabled is different from sent. A heartbeat failure does not change the watchdog exit code. Use --mode paper for a paper-only deployment.

Check and sync Norgate on a client

ACTIVE

Tests access and required snapshots. It can download files and update readiness metadata.

```
uv run python scripts/doctor_norgate_client.py
```

Example output - selected lines

```
[PASS] api healthz
[PASS] api token auth
[PASS] sync snapshots
[PASS] manifest hash validation
RESULT: PASS
```

FAIL: read the failing line before starting serve. This check does not prove broker connectivity. Use --overwrite only when you intend to replace an existing same-date snapshot.

Collect evidence for support

INSPECT

Creates a redacted baseline bundle and ZIP. Default collection does not tick, submit or reconcile.

```
.\scripts\collect_vps_debug_bundle.ps1 -Mode live -PodId pod_dv2_01
```

Expected output shape - paths vary

```
Bundle directory: ...
Bundle zip:      ...zip
```

Review any Non-zero commands line. If the service uses a custom DB or release root, pass the same -DbPath and -ReleasesRoot. Deeper doctor/probe options have extra side effects.

6. Useful extras and quick lookup

Keep this page for reports, the web view, common issues, tests and research commands.

Export orders for review

INSPECT

Writes an Excel trade sheet from the latest decision and order plan.

```
uv run python -m alpha.live.runner export_trade_sheet --mode paper --pod-id pod_dv2_01
```

Example output - selected lines

```
TRADE SHEET EXPORT
Orders: 2
VPlan: id=42 status=ready
File: ../trade_sheet.xlsx
```

If the output says decision intent only, there is no broker-sized VPlan yet. Exporting the sheet does not send orders.

Compare with a chosen backtest

INSPECT

Builds a comparison report; replace the pickle path with the intended reference.

```
uv run python -m alpha.live.runner compare_reference --mode paper --pod-id pod_dv2_01 `
--reference-strategy-pickle PATH_TO_REFERENCE.pkl --html
```

Example output - selected lines

```
Live vs Backtest Comparison
- Actual equity: 100100.0000
- Backtest equity: 100120.0000
- HTML report: ...
```

Before interpreting differences, align dates, capital, release, timing and accounting. Missing decision-time data means a reconstruction, not an exact replay. It may report no comparable VPlans.

Web view and common checks

```
uv run python -m alpha.live.dashboard_v3
# Open http://127.0.0.1:8080/paper
uv run python -m alpha.live.runner --help
```

Expected: a local web server and pod lifecycle view; --help prints commands and flags. Dashboard Operator Tools can also change state after confirmation.

Symptom -> first useful check

Missing/gray pod -> release mode, enabled flag, release root and correct DB.

Stale data -> snapshot time and Norgate client check.

Missing broker ACK -> show_vplan and broker connection; do not blindly resubmit.

Missed window -> inspect the reason; a late trade is a separate timing decision.

Tests and research - separate from live operations

```
uv run pytest tests/test_zero_share_fill.py -q
uv run python strategies/run_strategy.py strategy_mr_dv2
uv run python -m alpha.bench
```

Expected: a pass/fail test summary; a strategy report; and the research UI at <http://127.0.0.1:8765>. A passing test or backtest is not live readiness.

Sources: COMMANDS.md; LIVE_START_HERE.md; docs/live/LIVE_RUNBOOK.md; docs/live/DEBUGGING_RUNBOOK.md; docs/live/INSPECTOR_CONTRACT.md; current runner, scheduler and watchdog code. Examples were checked against source, not executed against the broker.